COMPUTER SCIENCE + COMPUTER ENGINEERING

Answer Key and Review Guidance

Separate solutions / Expanded Volume 04

Worked explanations for quizzes, problems, readiness checks, transfer tasks and module reviews.

100 lesson answers · 50 problem solutions · 25 review answers

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Contents

Module 16: Processor architecture and assembly 4

16.1: lesson quiz answers 4

16.2: lesson quiz answers 4

16.3: lesson quiz answers 5

16.4: lesson quiz answers 6

Module 17: Operating systems and resource management 8

17.1: lesson quiz answers 8

17.2: lesson quiz answers 8

17.3: lesson quiz answers 9

17.4: lesson quiz answers 10

Module 18: Concurrency and parallel programming 12

18.1: lesson quiz answers 12

18.2: lesson quiz answers 12

18.3: lesson quiz answers 13

18.4: lesson quiz answers 14

Module 19: Networks, storage and data services 16

19.1: lesson quiz answers 16

19.2: lesson quiz answers 16

19.3: lesson quiz answers 17

19.4: lesson quiz answers 18

Module 20: Engineering practice, security and maintainability 20

20.1: lesson quiz answers 20

20.2: lesson quiz answers 20

20.3: lesson quiz answers 21

20.4: lesson quiz answers 22

Gate D oral-review guide 24

Package 05 · Expanded Volume 04

Attempt each task before consulting this document. Alternative correct approaches are acceptable when they satisfy the stated contracts and make assumptions explicit. These are worked explanations, not evidence that the learner has completed the assignments.

The first three questions for each lesson reproduce the original key. The two application answers are expansion material. Code reference implementations are in `native/reference` and `python/reference`; they are separated from intentionally incomplete student files.

Readiness diagnostic answers

1. **What is wrong with returning a reference to a function-local object?**

   The object's lifetime ends at return, leaving a dangling reference.

2. **Does vector.reserve(100) create 100 live elements?**

   No; capacity and size are different.

3. **Why is ordinary signed integer overflow not a suitable wrapping model?**

   C++ does not define it as ordinary modular wrapping; use a deliberate checked or unsigned representation.

4. **Who owns the target of a raw pointer?**

   The pointer type alone does not say. The interface must identify an owner and valid lifetime.

5. **What must an exception-safe replacement preserve when validation fails?**

   Under a strong guarantee, the previously valid observable state remains unchanged and temporary resources are released.

6. **What does a successful compile fail to establish?**

   Link completeness, runtime correctness, resource safety, security, performance and satisfaction of application requirements.

7. **What is a directed dependency cycle and why reject it here?**

   A chain of prerequisites returns to a member; no simple prerequisite-first execution can start all members.

8. **How do you verify that a test actually detects a relevant defect?**

   Introduce a controlled mutation in a disposable copy and confirm the intended test fails; then restore the correct code.

9. **Why record build type with a timing result?**

   Optimization and instrumentation materially affect the executed program and its cost.

10. **Why cannot a raw C++ struct be assumed to be a portable network format?**

    Layout, padding, type widths and byte order require explicit encoding decisions.

11. **What is the difference between mean and tail latency?**

    Mean summarizes average duration; a tail statistic describes relatively slow observations and may reveal pauses the mean hides.

12. **What does an input bound need beyond a maximum list length?**

    Element types and ranges, total byte/work limits, failure behavior and the resource lifetime/ownership assumptions relevant to the operation.

Module 16: Processor architecture and assembly

16.1: lesson quiz answers

1. **How does an ISA differ from a microarchitecture?**

   The ISA is the software-visible contract; the microarchitecture is a particular implementation.

2. **What does a calling convention specify?**

   Argument/return placement, register preservation and related function-call rules.

3. **Why identify the exact ISA subset in a simulator?**

   Unsupported instructions or extensions cannot be assumed to behave like a complete implementation.

4. **What stack address contains the saved s0 after sp has been decremented by 16?**

   The adjusted sp plus 8 bytes. Relative to the original sp it is original_sp-8.

5. **What result should the unsigned 32-bit reference produce for [4294967295,2], and why not use signed overflow?**

   It produces 1 modulo 2^32. Ordinary signed overflow is not a portable wrapping contract in C++.

Independent transfer answer

A caller keeps an important value in s0. Your sum routine returns 23 but leaves s0 equal to 23. Is the call correct?

No. It violated the callee-preservation contract. Restore the saved s0 independently of returning the sum in a0; correct numerical output is only one part of ABI correctness.

Additional-problem solutions

16.P01. Write initialization, preservation and termination statements for the assembly accumulator.

Initially k=0, remaining=n and sum=0. Each valid load adds word k and advances k by one while decrementing remaining. At remaining=0, k=n and the accumulated low 32 bits cover the entire specified range.

16.P02. Remove the redundant ra save/restore from this leaf routine. Which obligations remain?

Preserve s0, restore sp, keep the required stack alignment, return in a0 and transfer control through the unchanged ra. Removing the frame entirely would also require avoiding or otherwise preserving s0.

16.P03. What additional evidence would support a claim that the RISC-V routine really executes correctly?

A supported assembler/link configuration and execution on a declared emulator or target with known inputs, output/register checks, and a documented memory environment. A host arithmetic result alone is insufficient.

16.2: lesson quiz answers

1. **What is a data dependency?**

   An operation requires a result produced by an earlier operation.

2. **Does higher clock frequency guarantee a faster program?**

   No. Work per cycle, stalls, memory and workload behavior also matter.

3. **Why can extra independent accumulators help?**

   They can expose parallel operations instead of one serial dependency chain, subject to resource and numerical constraints.

4. **A model says add latency is four cycles and issue rate is two adds per cycle. What bounds apply to eight dependent adds?**

   The dependency chain still requires at least 32 cycles; the four-cycle issue-count bound is weaker. More issue bandwidth does not remove true dependencies.

5. **Why is a checksum necessary but not sufficient for a benchmark?**

   It helps detect wrong results and make useful work observable, but does not establish that the timed work, compiler transformations and comparison conditions match the intended experiment.

Independent transfer answer

Four accumulators are slower on a 12-element list. What should be concluded?

Only that this implementation and environment did not improve that small workload. Setup, merging and compiler transformations may dominate. Retain the result and test a justified larger workload rather than changing the correctness contract.

Additional-problem solutions

16.P04. Estimate cycles for 1000 instructions at CPI 1.5 and a 3 GHz clock under a simple average model.

1500 cycles / 3,000,000,000 cycles per second = 0.5 microseconds. This is only as valid as the assumed CPI and excludes omitted effects.

16.P05. A branch-miss penalty is assumed to be 12 cycles and a trace has 25 misses. What extra term enters the toy model?

300 penalty cycles. Do not transplant this assumed penalty onto a real CPU without evidence.

16.P06. Explain a floating-point case where regrouping can change a sum.

With finite precision, a large positive value and its negative can cancel before adding a small value, preserving it; adding the small value to the large one first may round it away. The exact demonstration depends on the chosen representation.

16.3: lesson quiz answers

1. **What is spatial locality?**

   Accessing data near data that was recently accessed.

2. **How many 4-byte values fit in a hypothetical 64-byte line?**

   16, ignoring boundary misalignment and other storage.

3. **Does every first-level cache miss access DRAM?**

   No. Another cache level or other hierarchy component may satisfy it.

4. **In the two-line, sixteen-byte model, why do addresses 0 and 32 conflict?**

   Their memory blocks are 0 and 2; both map to index 0 but have different tags, so one replaces the other.

5. **Why does the benchmark traverse every stride residue?**

   To visit the same number of elements and obtain the same useful result as sequential and random traversal, rather than making the strided version do only a fraction of the work.

Independent transfer answer

Your timing curve slows at a particular array size. Does that prove the L2 size equals the array size?

No. Active index data, other allocations, translation, prefetching, scheduling and other cache levels can affect the transition. State it as a hypothesis and gather independent capacity or counter evidence.

Additional-problem solutions

16.P07. Trace addresses [0,16,0,16] in a one-line, sixteen-byte model.

All four accesses miss because the two memory blocks repeatedly replace each other.

16.P08. A loop uses 1,048,576 four-byte values and one eight-byte index per value. What active-array storage is represented?

4 MiB of values plus 8 MiB of indices = 12 MiB for that pair of arrays. Other allocated index variants and runtime storage still contribute to total memory.

16.4: lesson quiz answers

1. **What does a TLB cache?**

   Virtual-to-physical address translation information and associated access details.

2. **Does four-lane SIMD guarantee a fourfold speedup?**

   No. Memory, dependencies, overhead and other resources can limit improvement.

3. **Why test array lengths not divisible by vector width?**

   The remainder must be handled without out-of-bounds accesses or omitted elements.

4. **How many complete width-eight blocks and remaining elements occur for n=19?**

   Two complete blocks cover sixteen values, leaving a tail of three.

5. **Why can a TLB miss happen without disk access?**

   The required mapping can already exist in memory; the translation cache must be refilled, not necessarily the data page loaded from storage.

Independent transfer answer

The compiler refuses to vectorize an overlapping-buffer loop. Should the programmer simply assert nonaliasing?

Only if callers really satisfy that restriction and tests cover it. Otherwise change the algorithm or preserve the overlap semantics. A false aliasing promise invalidates the comparison.

Additional-problem solutions

16.P09. For a 4096-byte page, split address 8195 into page number and offset.

8195 = 2*4096+3, so page number 2 and offset 3.

16.P10. Design a test that distinguishes an omitted tail from a correct width-four loop.

Use five or seven elements with nonzero tail inputs and sentinel output initialization. Verify every expected value and unchanged sentinels outside the requested span.

Module-review answers

1. **Explain why a correct C++ result, a correct calling convention and a fast implementation are three different claims.**

   Language-level computation, binary-interface cooperation and measured performance have different contracts and evidence. None automatically establishes the others.

2. **A cache model predicts eight misses while a timing experiment slows much more than expected. What should happen next?**

   Check model scope, other memory traffic, translations, prefetching, dependencies and environment. Treat timing as observation and test hypotheses; do not force measured data to equal an incomplete model.

3. **A width-four loop reads four values on every iteration until i<n. What bug appears at n=5?**

   The iteration starting at i=4 reads/writes beyond the requested range unless masked or handled separately. Use a correct full-block condition and tail.

4. **What metadata must accompany an assembly or vectorization report?**

   Compiler/version, target/ABI, language mode, optimization flags, exact source and relevant assumptions such as alignment/aliasing.

5. **Design an equal-useful-work locality comparison.**

   Visit every element once for each pattern, verify the same result, generate indices outside timing, include index traffic in the stated boundary, vary working-set sizes and retain repeated trials.

Module 17: Operating systems and resource management

17.1: lesson quiz answers

1. **What do threads in one process typically share?**

   The process address space and many resources, while retaining separate execution state.

2. **What is a system call?**

   A controlled request from application code for an operating-system kernel service.

3. **Why might processes be preferable to threads for a risky tool?**

   Stronger isolation can contain faults, though communication and resource costs may be higher.

4. **Why is an argument list with shell=False useful here?**

   It launches the selected program with explicit argument boundaries rather than asking a shell to interpret user-controlled command text. It is not a complete sandbox.

5. **What is wrong with returning zero whenever a worker crashes?**

   Zero could be a valid result and would conceal failure. Use a distinct failure path and preserve useful diagnostics.

Independent transfer answer

A child process can still overwrite a file owned by the same account. Does that contradict process memory isolation?

No. Memory isolation and filesystem authority are distinct. The child may inherit permission to access the file even though it does not directly share the parent heap.

Additional-problem solutions

17.P01. Compare one shared-memory thread with one serialized subprocess for a mesh-analysis tool.

Threads avoid serialization and share storage but require synchronization and have a shared failure domain. A subprocess separates memory/faults more strongly but pays startup and communication costs and still needs permission/resource controls.

17.P02. Which observations should be recorded for a reproducible worker failure?

The input or minimized synthetic case, executable/source revision, command, environment, exit status, timeout policy, and sanitized diagnostic output.

17.P03. Does setting a three-second subprocess timeout guarantee completion within exactly three seconds?

No. It bounds the wait policy under documented API behavior, while process creation, termination and scheduling add overhead. Treat it as a failure-handling budget, not a hard-real-time guarantee.

17.2: lesson quiz answers

1. **Does reserving virtual address space imply all pages are resident?**

   No.

2. **Why may process memory usage stay high after freeing objects?**

   The allocator or runtime may retain storage for later reuse rather than returning it immediately to the OS.

3. **Does memory mapping eliminate storage latency?**

   No. Access can still trigger page faults and underlying I/O.

4. **Why does the empty-file case need an explicit branch?**

   Zero-length mappings are not portable across platforms; the digest of an empty byte sequence is well-defined without creating a mapping.

5. **Why should the mapping be closed before its resources are discarded?**

   The view has its own lifetime and may still expose storage. Explicit nesting prevents an asynchronous user from accessing a closed mapping.

Independent transfer answer

An allocator frees 1000 game entities but the process memory graph stays flat. Is that evidence of a leak by itself?

No. The allocator may retain reusable storage, and the graph may measure a different quantity. Track live ownership and allocations before inferring a leak from one aggregate measurement.

Additional-problem solutions

17.P04. A mapping begins on a page boundary and covers 12,289 bytes with 4096-byte pages. How many pages does it touch?

ceil(12289/4096)=4 pages. The last page contributes one byte; residency still depends on access and the system.

17.P05. Design an experiment that separates mapping setup from a complete scan.

Measure open/map/close separately from scanning an already established view; also measure end-to-end open/map/scan/close. Verify identical content, alternate order and preserve all trial boundaries.

17.P06. Why can a copied bytes(mapping) undermine an intended zero-copy experiment?

It allocates and copies the mapped contents, introducing additional memory and work. The mapping may still avoid another application-level read buffer, but that particular copy must be counted.

17.3: lesson quiz answers

1. **What is priority inversion?**

   Higher-priority work is delayed by lower-priority work through a shared-resource dependency.

2. **Does low average CPU use guarantee deadlines are met?**

   No. Bursts, blocking and scheduling delays can still cause misses.

3. **Why avoid blocking I/O in an audio callback?**

   Its unpredictable delay can exceed the callback deadline and cause audible interruption.

4. **A job released at 4 starts at 9, runs for 2 and has absolute deadline 10. What are waiting time and lateness?**

   Waiting time is 5; finish is 11; positive lateness is 1.

5. **Why is splitting a background task insufficient if all chunks still monopolize a shared lock?**

   The high-priority task remains blocked across the combined critical section. The repair must change the resource-holding or handoff structure, not only function boundaries.

Independent transfer answer

A performance report says average frame cost is 4 ms, so every 8 ms deadline must be met. Find the missing evidence.

An average does not bound individual completion times or blocking. Inspect per-job deadlines, release-to-completion latency and the long-tail/contended cases.

Additional-problem solutions

17.P07. Simulate jobs [(0,3,5),(1,2,4),(8,1,10)].

They run 0..3, 3..5 and 8..9. Positive lateness is 0,1,0; there is an idle interval from 5 to 8.

17.P08. Give one safety property and one conditional liveness property for an audio-data handoff.

Safety: no callback reads freed or partially published data. Conditional liveness: assuming the producer completes and scheduling continues, the callback eventually observes a published replacement. They require different arguments.

17.4: lesson quiz answers

1. **Does every successful write imply durable storage?**

   No. Data may remain buffered unless the required durability operations and guarantees are satisfied.
2. **Why retain an asynchronous I/O buffer until completion?**

   The operation may still read or write that storage after the initiating call returns.
3. **What should a save-recovery test inject?**

   Partial writes, malformed data, interruption points and replacement failures under an explicit failure model.
4. **What state is expected after the reference injects after_replace?**

   The new save is visible, even though an exception is reported. This is a post-commit response failure, not a rollback.
5. **Why create the temporary file in the destination directory?**

   It keeps the replacement within the same intended filesystem context and avoids a cross-filesystem rename assumption; the exact durability contract still needs platform-specific analysis.

Independent transfer answer

The caller times out waiting for an asynchronous save future. May it immediately reuse a mutable buffer that the worker might still read?

Not unless the API transferred or copied ownership so the old storage is no longer used. A timeout is not completion. Wait, cancel under a defined contract, or design an owned immutable snapshot.

Additional-problem solutions

17.P09. Create a failure table with pre-serialization, partial temporary write, before replacement and after replacement.

Invalid/pre-serialization work leaves the old destination; partial temporary write and before-replacement failures also preserve it; after replacement exposes the new valid destination. Test cleanup and explain power-loss exclusions separately.

17.P10. Why does fsync on a temporary file alone not justify a universal durable-save claim?

The destination-name update, directory persistence, filesystem ordering and device behavior also matter. State the actual platform operations and failure model instead of extending one call into an untested guarantee.

Module-review answers

1. **Distinguish a process boundary, a mutex and a future.**

   A process boundary separates an execution environment/address space; a mutex orders cooperating accesses; a future represents completion/result state. They are not interchangeable isolation guarantees.

2. **Why can an asynchronous save still cause a visible stall?**

   The caller may wait on its future, contend on shared locks, or exhaust a bounded queue; I/O still has latency even when performed elsewhere.

3. **What failure model does the reference save actually test?**

   Ordinary process exceptions at explicit stages in a trusted single-writer scratch directory. It does not simulate power loss, arbitrary filesystem corruption or hostile path races.

4. **How can low mean CPU usage coexist with missed audio deadlines?**

   Short bursts, long critical sections, scheduling delays and I/O dependencies can exceed an individual deadline despite a low long-term mean.

5. **A mapping digest disagrees with a streaming digest. Give a disciplined investigation order.**

   Confirm identical file/version and no concurrent writer, check length/offset/lifetime behavior, reduce to a small binary fixture, compare bytes, then investigate the implementations instead of asserting an API is unreliable.

Module 18: Concurrency and parallel programming

18.1: lesson quiz answers

1. **Can individually atomic reads/writes still form a logical race?**

   Yes. A multi-step invariant can be violated by interleaving even when each access is atomic.

2. **What must a mutex protocol protect?**

   The entire shared invariant and every conflicting access that participates in it.

3. **Why is "this thread usually runs first" insufficient?**

   Scheduling is not the required synchronization guarantee and can change across runs or systems.

4. **Why does snapshot() also lock?**

   It must participate in the same protocol to read a coherent gold/stock state and avoid conflicting unsynchronized access.

5. **Does the order of successful buyer threads need to match between test runs?**

   Not for the specified inventory invariant. Assert final state and valid operation effects, not an undocumented fairness or scheduling order.

Independent transfer answer

Both gold and stock are atomic integers. Is that enough to make a purchase across both fields transactional?

No. Atomicity of individual accesses does not compose automatically into a compound invariant. Use a complete synchronization algorithm, such as the single mutex in this reference.

Additional-problem solutions

18.P01. From gold=17 and stock=9, how many price-4 purchases can succeed?

Four, leaving gold=1 and stock=5; neither more threads nor more attempts creates additional funds.

18.P02. Give an invariant a test can check after every recorded completed transaction.

Gold and stock remain nonnegative; each success changes them by exactly the declared price and one item; each rejection preserves both. Records need a coherent observation protocol.

18.P03. Why is a sleeps-based ordering test a weak proof of synchronization?

Sleeping influences timing but does not create the required happens-before relationship or guarantee when another thread runs. Use locks, atomic publication, joins or other specified synchronization.

18.2: lesson quiz answers

1. **Why test a condition-variable predicate in a loop?**

   Wakeups can be spurious or another thread can consume the condition before the waiter proceeds.

2. **What does a consistent lock order prevent?**

   A common source of circular wait and deadlock.

3. **Why must shutdown notify blocked workers?**

   Without new work or a notification, they may never observe the stop condition.

4. **Why must closing a full queue notify producers as well as consumers?**

   A producer blocked for space must wake, observe closure and return a failure result even when no consumer will free space.

5. **What happens to work already removed by a consumer when close(false) runs?**

   It remains owned by that consumer. Discard affects only pending queue entries; stopping or rolling back active work requires another contract.

Independent transfer answer

A stop flag is set but an empty-queue consumer never wakes. What must change?

The stop/closed state must be part of the mutex-protected predicate and closure must notify the waiting consumers. The waiter must recheck that predicate rather than waiting only for data.

Additional-problem solutions

18.P04. List a valid shutdown order for one producer and two consumers.

Stop accepting new producer work, finish/join the producer or make its push observe closure, close with the chosen drain/discard policy, wake all waiters, then join consumers before destroying shared storage. Avoid holding their required locks while joining.

18.P05. Why can a notification arrive without making a consumer eligible to pop?

The wake can be spurious or another consumer may take the available item first. Only the protected predicate decides eligibility.

18.P06. Construct a deadlock involving a task that enqueues work into its own full bounded pool and then waits for it.

All workers can be occupied by parent tasks waiting to enqueue or wait for children while no worker remains to execute queued work. Avoid synchronous nested waits, reserve execution capacity or use a scheduler with a justified helping protocol.

18.3: lesson quiz answers

1. **Is volatile a C++ inter-thread synchronization mechanism?**

   No.

2. **What is false sharing?**

   Coherence contention caused by independent data sharing a cache line.

3. **Why is freeing a node difficult in lock-free structures?**

   Another thread may still hold or access the node; safe reclamation needs a separate protocol.

4. **Why is a relaxed counter acceptable in the padding benchmark but not a drop-in ready flag replacement?**

   The counter only tracks its own atomic value and is checked after joins. The ready flag is used to order access to a separate non-atomic payload, which requires the publication relationship.

5. **Does alignas(64) prove two counters cannot false-share on every target?**

   No. It establishes a C++ alignment/layout condition, not the target coherence granularity or all neighboring allocations.

Independent transfer answer

A developer clears ready and writes another payload immediately after publication. Does the original proof still apply?

No. Reuse introduces a new protocol. The producer needs evidence that the consumer is finished with the old payload, and the consumer needs a way to distinguish the intended publication. Use a handshake, queue or properly designed generation protocol.

Additional-problem solutions

18.P07. Identify the point at which the consumer may first read payload in the one-shot protocol.

After an acquire load observes the producer's true release publication, while the payload is still alive and no producer modifies it again.

18.P08. What extra protocol is needed before freeing a node reached through an atomic pointer?

Safe reclamation: evidence that no other participant can still access that node. Atomic pointer replacement alone does not invalidate all copies held by readers.

18.4: lesson quiz answers

1. **What is the ideal maximum speedup with a 25% serial fraction?**

   4x, since the parallel portion cannot eliminate the remaining 0.25 of runtime.

2. **Why can very small tasks reduce performance?**

   Scheduling, synchronization and bookkeeping can cost more than the useful work.

3. **What does a task dependency express?**

   A required ordering or data-availability relationship before a dependent task may execute.

4. **For a 20% serial fraction and four ideal workers, what is Amdahl speedup?**

   1/(0.2+0.8/4)=2.5. Actual overhead and contention can reduce it.

5. **Does the supplied executor implement work stealing?**

   No. It scans shared graph state for ready work. Work stealing is studied as an alternative and requires its own implementation and evidence.

Independent transfer answer

All workers execute parent jobs that synchronously wait for their own child jobs in the same bounded pool. What failure is possible?

Pool starvation or deadlock: no worker is available to execute the queued children. Represent dependencies in the scheduler rather than occupying every worker with blocking waits.

Additional-problem solutions

18.P09. Compute the span of A=3, B=2 after A, C=7 independent, and D=1 after B and C.

A-B-D is 6; C-D is 8. Span is 8 and total work is 13.

18.P10. How should a graph handle a failed asset-decode task whose upload depends on its output?

Record failure, do not publish a success output, and block or explicitly redirect the dependent upload to a validated fallback. Running it on an uninitialized result violates the dependency contract.

Module-review answers

1. **State a queue safety invariant and a conditional liveness claim.**

   Safety: size stays in [0,capacity] and no entry is read before acceptance. Conditional liveness: under continuing scheduling and the stated close/consumer policy, waiters eventually complete or observe closure. Fairness and task termination assumptions must be explicit.

2. **Why does join while holding a worker-needed lock risk deadlock?**

   The owner waits for the worker, while the worker waits for the owner's lock. Release shared resources before joining.

3. **Explain one-shot acquire/release publication without relying on x86 behavior.**

   Write payload, release-publish ready, acquire-read the corresponding publication, then read payload under the single-publication and lifetime rules. The argument is language-level ordering.

4. **Compute the ideal speedup for 10% serial work and four workers.**

   1/(0.1+0.9/4)=1/0.325, approximately 3.08, before real overhead.

5. **A failed graph node has descendants and an independent sibling. What does the supplied executor do?**

   Records the failure, blocks descendants whose inputs cannot be valid, and permits independent healthy work to complete. It does not forcibly interrupt arbitrary task bodies.

Module 19: Networks, storage and data services

19.1: lesson quiz answers

1. **Does TCP preserve application message boundaries?**

   No. It supplies an ordered byte stream.

2. **Does high bandwidth imply low latency?**

   No. Capacity and delay are different properties.

3. **What behavior must an application add when using UDP?**

   Whatever reliability, ordering, congestion control, validation and security its requirements need.

4. **What is the seven-byte wire frame for abc?**

   Hexadecimal 00 00 00 03 61 62 63: four length bytes and three ASCII payload bytes.

5. **Why close a decoder after an invalid length rather than scanning for another apparent prefix?**

   The protocol does not define resynchronization. Guessing can reinterpret attacker-controlled body bytes as a header and conceal corruption.

Independent transfer answer

Your server receives two commands in one recv call. Is TCP broken?

No. Multiple application messages can arrive in one byte-stream read. The framing parser must extract both according to their explicit lengths.

Additional-problem solutions

19.P01. A payload has 512 bytes. What is the frame size, excluding lower-layer headers?

516 bytes: four application length bytes plus 512 payload bytes.

19.P02. Give one parser test and one transport integration test that cover different risks.

Parser: split a concatenated frame stream at every byte position and verify exact emitted bodies. Transport: exchange a framed request on localhost and verify timeout, EOF and response handling.

19.P03. Why is TCP reliability not equivalent to authenticated player intent?

Transport delivery properties do not establish who should be allowed to perform a game action or whether its contents obey game rules. Authentication, authorization and validation remain separate.

19.2: lesson quiz answers

1. **Does a timeout prove the remote operation never happened?**

   No. The operation may have executed while the response was delayed or lost.

2. **Why validate a length before allocating its buffer?**

   An attacker or malformed file could request excessive memory or trigger overflow.

3. **What does idempotence mean?**

   Repeating an operation has the same intended effect as applying it once.

4. **Why reject bool values explicitly in a Python integer schema?**

   The application requires numeric integer inputs, not booleans; a broad isinstance(value,int) check alone would accept True and False.

5. **What does the reference do when result history is full?**

   It preserves existing records and rejects new unique operations. Existing identical replays remain meaningful; no silent eviction broadens the accepted workload.

Independent transfer answer

The client timed out and invents a new ID for the retry. What guarantee has it lost?

The store can no longer recognize that this is the same intended operation. Reusing the original ID and identical payload is necessary for the defined deduplication protocol.

Additional-problem solutions

19.P04. Give three limits that must be checked before launching the native worker.

Encoded request <=4096 bytes, list length 1..256, and each value an exact integer in 0..1000. Version, ID, field set and operation are also validated.

19.P05. Describe a retry policy with bounded amplification.

Reuse the same ID/payload, limit attempt count and total deadline, space attempts according to a documented backoff policy, and surface uncertain outcome instead of retrying forever.

19.P06. Why compare stored canonical work rather than only accepting any matching ID?

Otherwise accidental or malicious ID reuse with different work could return an unrelated result or overwrite the meaning of a committed operation.

19.3: lesson quiz answers

1. **What does a foreign key protect?**

   A specified relationship by requiring referenced values to exist, subject to its defined rules.

2. **Why can indexes slow writes?**

   The index structures must be maintained alongside the table changes.

3. **Why use parameterized SQL?**

   It keeps input data separate from SQL structure and prevents common injection mistakes.

4. **Why enable SQLite foreign keys on each connection?**

   The lab relies on explicit per-connection enforcement. Creating a REFERENCES clause without the required connection setting would not provide the tested behavior.

5. **Why use a composite uniqueness constraint on player_id and item_id?**

   It enforces one aggregate inventory row for each player/item pair, preventing duplicate state that the application would otherwise have to reconcile.

Independent transfer answer

A tiny-table query ignores a newly created index. Must the learner force index use to pass?

No. The goal is to explain the actual plan and workload, not force an optimizer choice. Verify correct results and compare a justified larger/selective workload while recording write and storage costs.

Additional-problem solutions

19.P07. Which query is naturally served by the leading player_id field of the composite key?

A lookup of all inventory rows for one player, or the exact player/item pair. An item-only query may need another index or a scan.

19.P08. Does a successful transaction context automatically close a Python sqlite3 connection?

No. Transaction handling and connection lifetime are separate. Explicitly close it or use an appropriate closing wrapper after its work is complete.

19.4: lesson quiz answers

1. **What does transaction atomicity mean?**

   The transaction's changes are treated as all-or-nothing under the database guarantee.

2. **Does a transaction in one database automatically include another service?**

   No. Cross-service coordination needs a separate protocol.

3. **What invariant should an item-transfer test preserve?**

   Total item quantity, valid nonnegative balances and the intended ownership changes.

4. **Why insert the replay record in the same transaction as the effect?**

   Otherwise an effect can commit without its deduplication record, allowing a repeated delivery to apply it again.

5. **Does a caught after_commit error imply total quantity changed incorrectly?**

   No. The transaction can have committed correctly while its response was lost. Inspect state and retry through the same identity protocol.

Independent transfer answer

The debit is committed in database X and the credit is sent to service Y, which times out. Is a local SQL transaction enough?

No. The effect crosses authority boundaries. It needs a separate coordination/recovery protocol and a clearly stated consistency model; a local rollback cannot undo an arbitrary remote effect.

Additional-problem solutions

19.P09. Give an assertion that distinguishes a rollback from an incorrectly half-applied transfer.

After an injected precommit failure, assert both original quantities and zero new replay records, not merely that the function raised an exception.

19.P10. What happens to an old retry if its operation record was deleted?

The service may treat it as new work. The guarantee must therefore specify retention, or the design must retain enough identity history to reject/replay it safely.

Module-review answers

1. **Why can a receive timeout leave the remote operation's outcome unknown?**

   The remote side may have committed while the response was lost or delayed. The timeout observes missing progress, not a guaranteed rollback.

2. **Which three records must be committed together for a deduplicated transfer?**

   Debit, credit and the operation identity/payload/result record. Separating their commits opens partial-effect or retry windows.

3. **Does a foreign-key definition alone establish enforcement in the SQLite exercise?**

   No. The reference explicitly enables it on every connection and tests direct invalid references.

4. **What happens when the same request ID is reused with a different payload?**

   The request is rejected as conflicting identity use; the previous committed effect and record remain intact.

5. **What is excluded by the loopback gateway's deployment scope?**

   Public exposure, per-user authentication, encryption, hostile-native-worker sandboxing and production availability guarantees. Those need a separate design and evaluation.

Module 20: Engineering practice, security and maintainability

20.1: lesson quiz answers

1. **What makes a requirement testable?**

   Observable behavior, conditions and an explicit acceptance criterion.

2. **Why record rejected alternatives?**

   They preserve the reasoning and make future reassessment more informed.

3. **Is completing planned tasks the same as satisfying requirements?**

   No. The implemented result must still meet the specified needs and constraints.

4. **Why name the retained database history in a replay requirement?**

   The guarantee depends on the record remaining available. A fresh or pruned database cannot silently inherit the old deduplication claim.

5. **What evidence is missing when a requirements matrix contains only implementation filenames?**

   Acceptance criteria, tests or review procedures, actual results, environment and any unresolved limitations. A file location alone does not establish behavior.

Independent transfer answer

Every planned programming task is complete, but the service accepts unbounded jobs. Is Gate D complete?

No. Task completion is not requirement satisfaction. Bounded admission is an observable system property and needs a conforming implementation and evidence.

Additional-problem solutions

20.P01. Rewrite "the server must never crash" as a bounded, testable requirement.

For a declared malformed-input corpus within the 4096-byte request limit, the process must return a classified failure, preserve stored results and remain able to accept a valid request. State the tested environment and do not extend this into universal crash freedom.

20.P02. Name one cost of using a subprocess for the tiny arithmetic operation.

Process creation and serialization can dominate useful work; output capture and failure supervision add complexity. The educational separation is deliberate, not a throughput recommendation.

20.P03. What should change in an ADR when a result contradicts the original assumption?

Add the new evidence, mark the old assumption as invalid or limited, reconsider alternatives and document the revised decision and migration cost rather than quietly rewriting history.

20.2: lesson quiz answers

1. **How does authorization differ from authentication?**

   Authentication concerns identity; authorization concerns allowed actions.

2. **Why is a game client a trust boundary?**

   The player controls that environment and can send incorrect or manipulated requests.

3. **Does encryption replace input validation?**

   No. Authenticated or encrypted input can still be malformed, malicious or contrary to game rules.

4. **Why does a worker-count limit alone not guarantee bounded backlog?**

   An executor may accept and queue more jobs than there are workers. Admission must separately bound queued-plus-running work.

5. **What additional fuzz cases reach beyond random-byte rejection?**

   Structure-preserving mutations of valid messages: boundary lengths, duplicate keys, wrong numeric types, unsupported versions and conflicting IDs.

Independent transfer answer

The service binds only to 127.0.0.1. Does that authorize every local caller?

No. Loopback restricts network exposure but does not establish per-user identity or permission. Keep the demo on a trusted workstation and do not claim local-user isolation.

Additional-problem solutions

20.P04. Write a threat-model row for untrusted length headers.

Asset: memory/availability. Entry: frame prefix. Failure: excessive allocation or indefinite wait. Controls: validate 1..4096 before body collection, absolute deadline, bounded chunks and classified rejection. Evidence: oversized-header and timeout tests.

20.P05. Write a residual-risk statement for the native worker.

Only an owner-selected trusted executable is supported. The subprocess wrapper does not contain hostile CPU/memory/output behavior or filesystem authority; supporting untrusted tools needs a separate sandbox and review.

20.P06. Why avoid calling a random parser test a penetration test of the operating system?

It exercises one local function with bounded inputs. It does not assess the OS, deployment, identity controls or every application path.

20.3: lesson quiz answers

1. **What is the purpose of dependency pinning?**

   To reduce uncontrolled version changes and support reproducible builds.

2. **What should identify a release artifact's origin?**

   Its source revision, build configuration, toolchain and dependency/asset versions.

3. **Why test a pipeline with an intentional failure?**

   To verify it actually blocks bad changes rather than merely displaying success regardless of checks.

4. **Why does the package manifest exclude its own hash?**

   Including a digest of the complete manifest in itself creates a circular definition. The exclusion is explicit rather than hidden.

5. **Does the same source hash guarantee identical executable bytes on two compilers?**

   No. Toolchain, flags, libraries, target and build metadata also affect the artifact. Reproducible procedure and bitwise reproducibility are distinct claims.

Independent transfer answer

The pipeline printed a failed test but returned exit code zero and created a release. Does it satisfy the acceptance rule?

No. Its control flow ignored failure. Propagate nonzero status and create the new release only after every required stage succeeds.

Additional-problem solutions

20.P07. List five facts needed to identify the origin of a tested native artifact.

Source revision or file hashes, compiler/version and target, build configuration/flags, dependency versions, and test/asset inputs or hashes. Also record the exact build commands.

20.P08. Explain why adding a checksum file to a download does not automatically authenticate it.

An attacker able to replace both can make them agree. Authenticity requires a trusted distribution channel or signature/key relationship in addition to content hashing.

20.4: lesson quiz answers

1. **Does public availability imply permission to redistribute?**

   No. Copyright and license terms still govern use.
2. **Why include accessibility in acceptance criteria?**

   Otherwise a technically functioning feature may remain unusable for part of the intended audience.
3. **What should a blameless retrospective seek?**

   Contributing system/process conditions and practical improvements rather than personal blame.
4. **Why must UNKNOWN_OUTCOME have different recovery instructions from INVALID_REQUEST?**

   A timed-out operation may already be committed; retrying with a new identity can duplicate the intended effect. Invalid input must instead be corrected before admission.
5. **What makes a retrospective action testable?**

   A named change, owner or responsibility, and observable acceptance evidence, such as a shutdown test that previously failed and now passes.

Independent transfer answer

An optimization removes clear errors and silently retries forever. Is higher throughput sufficient justification?

No. It violates bounded recovery and operator-understanding requirements. Evaluate performance together with resource limits, correctness and usability.

Additional-problem solutions

20.P09. Write three actionable improvements after a cancellation incident.

Add queued/running cancellation tests with controlled synchronization; document the commit-versus-cancel ordering; and update operator messages to distinguish pending cancellation from already committed work. Give each a reviewable acceptance check.

20.P10. What must an honest portfolio say about using the reference service?

Identify reused reference/support code, describe the student's independent design and implementation changes, show original tests and measurements, and avoid claiming production deployment or security guarantees not demonstrated.

Module-review answers

1. **What distinguishes the original curriculum requirements from the new ForgeQueue design?**

   The source fixes learning outcomes, original labs/quizzes and Gate D evidence. ForgeQueue is an authored worked implementation choice with explicit limits, not a replacement for every allowed student design.

2. **Why must pipeline failure injection precede trusting a green release script?**

   A script can display failures yet return success or publish stale output. Injection checks whether failure actually blocks creation of the new artifact.

3. **Give one claim that code tests cannot fully establish.**

   Accessibility/usability for intended users, completeness of a threat model, correctness of all memory-model arguments, or suitability of an architecture under untested conditions needs further review and evidence.

4. **Why is rejecting new unique jobs when history is full sometimes the correct behavior?**

   It preserves a stated deduplication retention guarantee and a storage bound. Silent eviction would change replay semantics; a production retention design needs explicit additional rules.

5. **What must a Gate D handoff let a reviewer reproduce?**

   A clean build and test run, normal processing, failure/recovery cases, ownership and replay arguments, workload measurements and their limitations; plus an independently explained student change.

Gate D oral-review guide

Bytes to result: explain length framing, strict schema validation, admission capacity, the trusted native command, checked result, transactional replay record and response. TCP delivery is not durable application commitment.

Race-free native reasoning: identify every shared location, conflicting access and ordering relationship. Mutexes protect compound invariants; release/acquire publication is one-shot and observes the corresponding release. Independent task bodies need disjoint writes or additional synchronization. A sanitizer result supplements this argument but does not replace it.

Cancellation versus commitment: the reference serializes the cancel flag and database-commit boundary with the service mutex. Before-commit cancellation suppresses the result; after-commit cancellation does not undo it. A gateway timeout can leave the outcome unknown to its caller.

Duplicate effects: the request identity, canonical payload and result are stored together. Same identity with different work is a conflict. A repeated computation can occur, while duplicate durable rows/effects are prevented under the retained-history model. Deleting replay records changes that guarantee.

Performance: retain raw trials and the declared measurement boundary. Separate native compute, subprocess startup, queue wait, serialization and storage. A fast sum-of-squares worker does not imply a fast whole service. There is no mandatory speedup factor.

Release: demonstrate a clean successful run and a deliberately failed test that prevents a new artifact. Explain that hashes support byte-integrity checks against a trusted manifest, not authorship authentication.

Limitations: no public-service security, arbitrary-code sandbox, crash-proof hardware storage claim, hard-real-time guarantee, full RISC-V implementation or cross-platform execution claim is provided by these demonstrations.